

Summary of Post-Training (09/21/2026)

Team Members: Arihan Iyer (**imarihan**), Jatin Shihora (**jatinsh**), Kushagra Mishra (**kushm**), Rohit Puthucode (**rohitps**)

1. Research Direction of the Field

Reinforcement learning (RL) post-training is the computational bottleneck of modern LLM development, requiring systems to orchestrate phases with conflicting resource profiles: compute-bound training, memory-bandwidth-bound autoregressive decoding, and non-deterministic environment execution. Early general-purpose RL frameworks (such as RLlib) relied on centralized controllers that incurred massive dispatch latency when scaled to LLMs. Conversely, first-generation LLM alignment systems (such as DeepSpeed-Chat and early OpenRLHF) adopted rigid, monolithic multi-controller architectures that hardcoded collective communication directly into worker execution loops, locking workflows into brittle, colocated pipelines on homogeneous clusters.

The lecture papers trace the architectural evolution from single-turn RLHF to disaggregated, multi-turn agentic systems: **HybridFlow** bridged the gap between flexibility and performance by introducing a hybrid control plane alongside efficient weight resharding between generation and training layouts. It established the blueprint for modern open-source RL runtimes (becoming the foundation of *veRL*).

As workloads shifted toward agentic tasks with multi-turn tool use, long-tail environment latencies, and divergent prefill/decode ratios, **RollArt** advanced the field by pioneering an asynchronous, disaggregated architecture that routes decoupled tasks to specialized hardware tiers and serverless compute to absorb stragglers.

This progression from synchronous, colocated orchestration (*HybridFlow*) to asynchronous, heterogeneous disaggregation (*RollArt*) directly paved the way for current resource-management frameworks like *Libra* (complexity-aware trajectory routing) and *ROSE* (harvesting elastic serving clusters for rollout efficiency).

2. Lecture Papers

Paper 1: [*HybridFlow: A Flexible and Efficient RLHF Framework*](#)

What is the problem of this paper?

The main challenge in RLHF is coordinating multiple LLMs, such as the actor, critic, reference, and reward models, even though each has different workloads. This makes RLHF expensive and difficult to optimize at large model sizes. The paper identifies three main problems: **Coordination**, where systems must choose between flexible but slower designs and efficient but tightly coupled ones; **Parallelism Mismatch**, where training and generation need different parallelism strategies, creating communication and resharding overhead when they share the same actor weights; and **Model Placement**, where the best placement depends on the model size and number of GPUs, so a fixed choice can cause GPU idle time and extra communication. Existing systems make different trade-offs, leading to issues such as high resharding costs, redundant model copies, GPU underutilization, and limited support beyond PPO. The paper's main insight is that these problems should be handled separately: the hybrid approach keeps coordination flexible across models while keeping execution efficient within each model.

What is the solution of this paper?

Instead of tightly coupling everything together, **HybridFlow** solves the three problems by separating coordination, model computation, and resource planning. Its hybrid programming model uses a single controller to coordinate across models while using multiple controllers within each model to distribute computation across GPUs, keeping coordination flexible without adding large overhead. The **3D-HybridEngine** lets actor training and generation use different parallelism strategies on the same GPUs, reducing memory duplication and resharding communication. **Auto-mapping** searches different model placements and GPU allocations and selects the configuration with the lowest estimated iteration time, allowing placement to adapt to the model size and available GPUs. The key idea is to separate computation from coordination while still keeping them in one framework. This gives HybridFlow both flexibility and efficiency, although it adds system complexity and relies on workloads being predictable enough for offline mapping.

How is the solution evaluated?

HybridFlow was evaluated on a cluster of 16 physical nodes hosting 128 NVIDIA A100-80GB GPUs. Experiments benchmarked the **PPO, ReMax, and Safe-RLHF** algorithms on **Llama-family** models scaling from 7B to 70B. Compared against baselines DeepSpeed-Chat, OpenRLHF, and NeMo-Aligner, the evaluation measured end-to-end throughput in tokens per second, scaling efficiency, placement optimality, and actor train-to-generation weight resharding latency. HybridFlow outperformed all baselines across every tested scale, achieving **1.53×~20.57×** overall speedup, sustaining **66.8%** strong scaling efficiency up to **128 GPUs**, and reducing transition overhead by up to **89.1%** on 70B models. An additional valuable experiment could have evaluated dynamic KV-cache memory pressure and fragmentation under longer context lengths.

Overall, what are 1–2 shortcomings of this paper?

The system is designed primarily for **homogeneous** GPU nodes, lacking the flexibility to natively disaggregate distinct compute and memory workloads across specialized hardware. While HybridFlow efficiently delegates matrix computation, its architecture suffers from bottlenecks in dataflow. The overall dataflow: including dataset loading and intermediate tensor aggregation is governed by a single centralized controller, creating **severe I/O serialization** particularly in long-context scenarios.

Paper 2: [*RollArt: Disaggregated Multi-Task Agentic RL Training at Scale*](#)

What is the problem of this paper?

Agentic reinforcement learning trains language models by repeatedly interacting with environments creating a pipeline of very different workloads. Text generation has both **compute-heavy** and **memory-heavy** phases, environment execution can be unpredictable and slow, reward scoring is lightweight but bursty, while training needs powerful GPUs. No single hardware type fits all these needs, yet existing systems either put everything on one cluster or separate the workloads only roughly. This causes problems such as slow environments stalling others, reward GPUs sitting mostly idle, and expensive weight transfers between clusters. These mismatches make existing systems struggle as workloads become more multi-turn and agentic.

What is the solution of this paper?

RollArt breaks the agentic training pipeline into smaller pieces and matches each one to the hardware best suited for it. Generation requests are routed to different GPU types depending on whether they are more compute-heavy or memory-bandwidth-heavy, while environments run independently per trajectory so one slow environment does not hold back the others. Reward scoring is moved to elastic **serverless infrastructure** instead of keeping dedicated GPUs idle. Training and rollout also run concurrently on separate hardware pools, with a limit on how outdated a trajectory can be before it is discarded. The system handles the scheduling, reward jobs, and **weight synchronization** automatically. Overall, this allows RollArt to improve training speed across different model sizes and operate at production scale, including a week-long run on a cluster with thousands of GPUs.

How is the solution evaluated?

The authors evaluate RollArt using **Qwen3-8B, 14B, and 32B** across several agentic RL tasks: software engineering, web interaction, games, and mathematics. They compare it with synchronous approaches and asynchronous systems such as One-off and AReaL, measuring training time and throughput. RollArt achieves about **1.31× to 2.05×** lower step time than the main baselines and **2.65× to 4.58×** higher throughput than the basic synchronous system. The authors also run ablations to measure the contribution of each major idea: hardware selection, independent trajectory execution, serverless reward computation, controlled staleness, and asynchronous weight transfer. Finally, they test RollArt on a production workload using more than 3,000 GPUs and a model with hundreds of billions of parameters. This deployment ran for a week and showed that the system could operate reliably at large scale.

Overall, what are 1–2 shortcomings of this paper?

One limitation is that hardware placement is still mostly configured manually. RollArt allows the user to specify which hardware is appropriate for different tasks, but it does not fully automate this decision or continuously adapt it when the workload changes. The authors identify **online profiling** and **dynamic routing** as a possible future improvement. A second limitation is the tradeoff introduced by asynchronous training. RollArt allows training to use slightly older trajectories so that the system does not have to wait, but allowing data to become too old could affect training stability. The paper finds that a staleness limit of one works well for its workloads, but it does not establish that this choice will be optimal for every workload. Also, the production experiment shows that once the **major synchronization problems** are reduced, waiting for enough trajectories in the SampleBuffer can itself become a significant bottleneck.

3. Lecture Discussions

How does the definition of a “trajectory” differ between the two papers?

In HybridFlow, a trajectory is a static, linear pass from prompt to response to reward/critic scoring, with predictable execution boundaries, which is suited for synchronized batch schedules. In RollArt, a trajectory is an interactive, non-deterministic loop of action to sandbox execution to observation, characterized by heavy-tailed environment latencies and dynamic, multi-step execution graphs.

Colocation vs. Disaggregation: Which paradigm dominates, and can they be combined?

Colocation maximizes utilization on homogeneous clusters, while disaggregation matches heterogeneous

hardware to different workload demands, such as compute-bound prefill and memory-bandwidth-bound decoding. They are complementary: a macro-level router can dispatch tasks across specialized pools, while a micro-level engine can optimize execution within each homogeneous cluster.

How do the systems handle scaling and the dynamic shifts that occur as agents learn?

HybridFlow's static offline search can become brittle with large context windows as intermediate data grows and may cause out-of-memory issues. RollArt instead assigns tasks to different hardware tiers at runtime. However, neither system provides an online scheduler that can rebalance resources when the workload shifts during training, for example, from compute-heavy runs to long, memory-bandwidth-bound decoding.

What are the hidden trade-offs of bounded staleness (α) and zero idle time?

Zero idle time means non-blocking execution, not necessarily 100% hardware utilization. Bounded staleness ($\alpha \geq 1$) trades some on-policy accuracy for throughput. Dropping stale trajectories can also create data selection bias, because short tasks finish quickly and contribute to updates, while difficult, long-horizon trajectories are more likely to be aborted. Token throughput alone may therefore hide an impact on end-to-end convergence.

4. Proposal for Follow-Up Work

Recent frameworks mitigate agentic RL resource fragmentation by routing based on task complexity (Libra) or harvesting idle serving GPUs (ROSE). However, multi-turn environments suffer from **stateful fragmentation**: an agent's context and compute demands can explode mid-rollout. Currently, neither approach can migrate active, long-context trajectories across hardware tiers without aborting the session and triggering expensive KV-cache recomputation.

We propose bridging complexity-aware scheduling via mid-trajectory migration. By monitoring live prefill-to-decode ratios and context growth, the framework detects when an agent on a harvested GPU crosses a compute-intensity threshold, then triggers a direct KV-cache transfer to a high-tier node. This dynamically matches hardware to evolving reasoning phases without redundant computation.

A possible issue is inter-cluster network saturation. Migrating KV-caches for large models at scale produces massive RDMA bursts. Simultaneous migrations could severely congest the fabric, stalling RL training loops and degrading the latency of co-located production serving workloads.

5. Reflection on AI Usage

1) When prompted to act as a "skeptical reviewer," the AI over-optimized for finding flaws and unfairly penalized the authors for the historical infrastructure limitations of the NeMo-Aligner baseline.

2) When I asked the AI to identify the most important problem, it called the programming-model mismatch the "root cause." After I asked whether solving it would eliminate transition and placement problems, it reconsidered and acknowledged that those problems still require separate optimization.